

1 Summary

2 Modern scientific research requires consistent document preparation, re-
3 producible computation, unified data management, and transparent figure
4 provenance—yet researchers rely on fragmented tools that increase cognitive
5 load and introduce irreproducibility. We present SciTeX, an AI-native, mod-
6 ular research automation platform providing four integrated modules: Writer
7 for structured LaTeX manuscript preparation with containerized builds; Scholar
8 for literature management with automated metadata extraction; Viz for
9 publication-quality figure generation with embedded provenance; and Code
10 for reproducible computation with GPU and container support. Each mod-
11 ule operates independently while sharing a unified project structure through
12 lightweight symlinks and global identifiers, enabling cross-module traceabil-
13 ity. SciTeX reduces the activation energy required for rigorous, reproducible
14 science while maintaining compatibility with traditional workflows. The plat-
15 form is open source and follows FAIR principles. This manuscript demon-
16 strates SciTeX’s self-descriptive capabilities by being prepared within the
17 platform it documents.

18 Highlights

- 19 • SciTeX integrates manuscript writing, figure generation, literature man-
20 agement, and computation in a unified platform
- 21 • Modular architecture enables independent use of each component while
22 providing synergy through integration
- 23 • Containerized compilation ensures reproducible manuscript output across
24 all operating systems
- 25 • Embedded metadata enables full provenance tracking from figures and
26 results back to generating code
- 27 • AI-native design provides integration points for LLM-assisted research
28 workflows

SciTeX: An AI-Native Modular Platform for Reproducible Scientific Research

Yusuke Watanabe^{a,*}

^a*SciTeX.ai, Tokyo, Japan*

Keywords: reproducible research, scientific workflow automation, data science platform, AI-native tools, figure provenance, FAIR principles

~ 8 figures, 3 tables, 136 words for abstract, and 3056 words for main text

1. Introduction

Scientific workflows increasingly span multiple heterogeneous environments: personal laptops, HPC systems, cloud servers, and containerized pipelines [? ?]. Traditional manuscript preparation tools remain fragmented, requiring manual management of figures, references, code execution, and version control. These inconsistencies lead to irreproducible results, duplicated effort, and high cognitive load that distracts researchers from their primary scientific objectives [?].

The preparation of scientific manuscripts involves numerous technical challenges that extend beyond the intellectual task of communicating research findings. Researchers must navigate complex typesetting systems, manage multiple document versions, coordinate figures and tables across formats, and ensure reproducible compilation environments [?]. Furthermore, maintaining consistency between computational analyses, their visual representations, and the manuscript text requires constant manual synchronization—a process prone to error and version drift [?].

Traditional approaches to manuscript preparation typically rely on local LaTeX installations, where specific versions of packages and compilation

*Corresponding author. Email: y.watanabe@scitex.ai

55 tools can vary significantly across machines and over time. This variability
56 creates reproducibility challenges, particularly in collaborative environments
57 where multiple authors work on different systems [? ?]. Similarly, figure
58 generation often occurs in isolation from the manuscript, with researchers us-
59 ing various plotting libraries without standardized metadata or provenance
60 tracking. Literature management frequently depends on external tools that
61 operate independently from the writing environment, leading to citation in-
62 consistencies and manual bibliography maintenance.

63 Existing solutions have addressed individual aspects of this problem.
64 Cloud-based platforms such as Overleaf provide consistent compilation en-
65 vironments but require continuous internet connectivity and separate figure
66 generation workflows. Zotero and Mendeley excel at citation management
67 but lack integration with manuscript preparation and computational analy-
68 sis. Jupyter notebooks and Binder enable reproducible computation but pro-
69 vide limited support for publication-quality figures or structured manuscript
70 writing [?]. Manubot pioneered open collaborative writing with auto-
71 mated citation processing [?], while Pandoc Scholar demonstrated agile
72 multi-format document generation [?]. SigmaPlot and Origin offer so-
73 phisticated visualization capabilities but operate as standalone applications
74 without provenance tracking or manuscript integration.

75 The fundamental challenge lies in unifying these disparate tools while
76 maintaining the flexibility that researchers require [?]. The solution must
77 accommodate diverse content types, multiple output documents, and vary-
78 ing journal requirements while ensuring a single source of truth for shared
79 elements. Additionally, modern research increasingly involves AI-assisted
80 workflows for code generation, literature review, and manuscript revision—
81 capabilities that require native integration rather than ad-hoc addition [? ?
82].

83 Version control systems, particularly Git, have emerged as powerful tools
84 for managing research workflows [? ?], enabling transparent tracking of
85 changes and facilitating collaboration [? ?]. However, effectively combin-

86 ing Git with LaTeX, figure generation, and literature management requires
87 significant technical expertise that many researchers lack [?].

88 Containerization technologies such as Docker have transformed software
89 reproducibility [? ? ?], and several frameworks have emerged to leverage
90 containers for reproducible research [? ?]. The Rockerverse ecosystem
91 demonstrates how containers can support reproducible R-based workflows [?
92], while tools like CREDO simplify Docker configuration for bioinformatics [?
93]. End-to-end templates for reproducible analysis and manuscript writing
94 have been proposed [? ?], yet comprehensive integration across all research
95 activities remains elusive.

96 SciTeX addresses these challenges through a unified, modular, AI-native
97 platform that allows researchers to: write manuscripts in a structured LaTeX
98 environment with containerized compilation; manage citations and PDFs
99 with consistency and automated metadata extraction; generate publication-
100 quality figures with embedded provenance metadata; and execute analyses in
101 controlled environments with synchronized results [?]. Importantly, SciTeX
102 maintains bidirectional low migration cost: each module functions indepen-
103 dently, providing value without requiring adoption of the entire ecosystem,
104 while synergy across modules enables a strictly superior integrated workflow.

105 The platform follows principles articulated in seminal works on repro-
106 ducible research [? ? ?] while incorporating modern infrastructure for con-
107 tainerized compilation [? ?], workflow automation [? ?], and FAIR data
108 practices [?]. By embedding reproducibility infrastructure into everyday
109 research tools, SciTeX makes rigorous practices the path of least resistance
110 rather than an additional burden [? ?].

111 Many researchers lack unified tools to integrate writing, computation,
112 visualization, and literature management within a single, reproducible en-
113 vironment. Graduate students and early-career researchers often spend dis-
114 proportionate time learning fragmented toolchains rather than conducting
115 research. Computational scientists require reproducible pipelines that con-
116 nect analysis code to manuscript figures without manual intervention [?].

117 AI-augmented research workflows demand native integration points for auto-
118 mated assistance. SciTeX provides this missing infrastructure, reducing the
119 activation energy required for rigorous, reproducible science while democra-
120 tizing access to advanced computational workflows for researchers unfamiliar
121 with Linux, Git, or containers.

122 This manuscript is prepared within SciTeX Writer, referencing figures
123 generated in SciTeX Viz, bibliography managed in SciTeX Scholar, and com-
124 putational environments controlled by SciTeX Code—thereby making the
125 manuscript a live demonstration of the platform’s integrated capabilities.

126 2. Methods

127 2.1. System Architecture

128 SciTeX consists of a Django-based cloud platform (SciTeX Cloud) and
129 a Python package (v2.4.1+) installable via `pip install scitex`. The ar-
130 chitecture employs lazy module loading via a custom `_LazyModule` wrapper,
131 enabling fast imports while providing access to over 50 specialized mod-
132 ules. This modular design follows best practices for scientific Python appli-
133 cations [? ?]:

```
134 import scitex as stx
135 stx.plt          # Publication plotting
136 stx.io           # Universal file I/O (30+ formats)
137 stx.scholar      # Literature management
138 stx.session      # Experiment lifecycle
139 stx.vis          # Visual figure specifications
140 stx.writer       # Manuscript preparation
```

141 Each module operates independently while sharing a unified project struc-
142 ture through lightweight symlinks. Global identifiers (SHA256 hashes) ensure
143 traceability between figures, code, bibliography entries, and manuscript con-
144 tent, enabling reverse lookup capabilities (e.g., “which script generated this
145 figure?”). This approach addresses concerns raised in discussions of figure
146 provenance and data reproducibility [? ?].

147 *2.2. Session Decorator for Reproducible Experiments*

148 The `@stx.session` decorator provides automatic experiment lifecycle
149 management, implementing principles from reproducible research frameworks [?
150 ?]:

```
151 import scitex as stx
152
153 @stx.session
154 def analyze(data_path: str, threshold: float = 0.5):
155     '''Analyze data file.'''
156     data = stx.io.load(data_path)
157     result = process(data, threshold)
158     stx.io.save(result, "output.csv")
159     return 0
160
161 if __name__ == '__main__':
162     analyze() # CLI: python script.py --data-path x --threshold 0.7
```

163 The decorator automatically: (1) generates CLI argument parsers from
164 function signatures with type hints, (2) initializes session directories with
165 timestamped run IDs, (3) injects global variables (`CONFIG`, `plt`, `COLORS`,
166 `rng_manager`, `logger`), (4) manages random state for reproducibility, and
167 (5) handles cleanup and optional notifications on completion. This design is
168 informed by the “good enough practices” paradigm [?] and project manage-
169 ment tools for team science [?].

170 *2.3. Publication-Quality Plotting (`stx.plt`)*

171 The `stx.plt` module wraps matplotlib with automatic configuration from
172 `SCITEX_STYLE.yaml`:

```
173 import scitex.plt as splt
174
175 fig, ax = splt.subplots(axes_width_mm=40, axes_height_mm=28)
```

```

176 ax.plot([1, 2, 3], [1, 4, 9])
177 ax.set_xyt("Time", "Value", "Analysis Results")
178 stx.io.save(fig, "figure.png") # Exports PNG + JSON + CSV

```

179 Key features include: (1) automatic style application on import (font
 180 sizes, line widths, spine visibility), (2) `FigWrapper` and `AxisWrapper` classes
 181 providing `set_xyt()` for simultaneous x-label, y-label, and title setting, (3)
 182 automatic CSV export of underlying plot data alongside figures, (4) JSON
 183 sidecar metadata encoding axis bounds, tick locations, panel geometry, and
 184 color palettes, and (5) `stx.plt.load()` for figure reconstruction from saved
 185 metadata. This triple-export approach (image, metadata, data) addresses
 186 the reproducibility challenges identified in computational figure generation [?
 187 ?].

188 Style values can be customized via YAML configuration, environment
 189 variables (`SCITEX_PLT_FONTS_AXIS_LABEL_PT=8`), or direct function param-
 190 eters.

191 2.4. Universal File I/O (*stx.io*)

192 The `stx.io` module provides format-agnostic file operations supporting
 193 over 30 formats, following the principle of unified interfaces for data man-
 194 agement [?]:

```

195 import scitex as stx
196
197 # Automatic format detection
198 data = stx.io.load("data.csv")      # DataFrame
199 config = stx.io.load("config.yaml") # Dict
200 model = stx.io.load("model.pkl")    # Python object
201 arrays = stx.io.load("data.h5")    # HDF5 exploration
202
203 # Unified save interface
204 stx.io.save(df, "output.csv")

```

```

205 stx.io.save(fig, "figure.png")      # PNG + JSON + CSV
206 stx.io.save(config, "config.yaml")

```

Supported formats include: CSV, JSON, YAML, Pickle, HDF5, Zarr, NumPy arrays, PyTorch tensors, images (PNG, JPG, SVG, PDF), audio, video, and MATLAB files. The module includes **H5Explorer** and **ZarrExplorer** for interactive hierarchical data navigation, plus configurable caching for frequently accessed files.

2.5. Literature Management (*stx.scholar*)

The Scholar module provides multi-source literature search and management, integrating capabilities typically found in standalone reference managers:

```

216 from scitex.scholar import Scholar
217
218 scholar = Scholar()
219 papers = scholar.search("deep learning", year_min=2022)
220 papers.save("bibliography.bib")

```

The module integrates with multiple metadata engines (CrossRef, Semantic Scholar, OpenAlex) via the **ScholarEngine** interface. Features include: automatic DOI detection and metadata extraction from PDFs, **ScholarPDFDownloader** for paper acquisition, **ScholarLibrary** for local storage management, and **Paper/Papers** classes with filtering and export capabilities. This design complements tools like Manubot [?] by providing direct integration with manuscript preparation.

2.6. Cloud Platform Architecture

The Django 5.1-based cloud platform provides web interfaces for all modules, following principles of continuous integration for scientific publishing [?]:

232 **Code App:** Browser-based Python execution with real-time output stream-
233 ing, file upload/download, and workspace management.

234 **Viz App:** Canvas-based figure editor using Fabric.js 5.5.2 for interactive
235 element manipulation, with JSON specification import/export for program-
236 matic control.

237 **Writer App:** Structured LaTeX manuscript preparation with container-
238 ized compilation (Docker/Apptainer), section-separated authoring, and au-
239 tomatic figure format conversion.

240 **Scholar App:** Literature cards with abstract preview, PDF access,
241 metadata editing, and direct citation insertion into Writer documents.

242 The platform employs TypeScript for frontend logic, PostgreSQL for data
243 persistence, and Docker for containerized compilation ensuring consistent
244 output across environments.

245 2.7. Containerized Compilation

246 Writer module compilation leverages containerization for reproducibility,
247 implementing guidelines for Docker-based scientific workflows [? ?]:

248 **Multi-Engine Support:** Tectonic [?] for ultra-fast incremental builds
249 (1–3 seconds), latexmk for reliable industry-standard compilation (3–6 sec-
250 onds), and traditional 3-pass compilation for maximum compatibility.

251 **Environment Isolation:** Docker and Apptainer/Singularity containers
252 encapsulate specific TeX Live versions and all required packages, eliminat-
253 ing host system dependencies and guaranteeing identical PDF output across
254 macOS, Linux, Windows, and HPC environments [? ?].

255 **Version Tracking:** Git integration [?] with automatic latexdiff gener-
256 ation facilitates peer review processes [?].

257 2.8. AI-Native Workflow Integration

258 SciTeX provides native integration points for AI-assisted workflows, an-
259 ticipating the growing role of AI in scientific research [? ?]:

260 **Structured APIs:** JSON-based figure specifications, typed function sig-
261 natures for CLI generation, and metadata formats amenable to automated
262 analysis enable LLM-assisted code generation and figure improvement.

263 **Workflow Hooks:** The session decorator and modular architecture pro-
264 vide hooks for AI-assisted revision, automated citation recommendation, and
265 semantic search across analyses and literature.

266 **Research Co-pilot Vision:** The architecture supports future devel-
267 opment of automated analysis pipelines, figure generation, and manuscript
268 drafting while maintaining researcher oversight and scientific integrity [?].

269 2.9. Implementation Details

270 The system supports deployment on local machines, cloud servers (AWS,
271 GCP, Azure), and self-hosted NAS environments. Key technical specifica-
272 tions:

- 273 • Python 3.12+ with GPU acceleration support (CUDA 11.x/12.x)
- 274 • Django 5.1 with PostgreSQL backend
- 275 • TypeScript frontend with webpack bundling
- 276 • Docker/Apptainer containerization for reproducible execution [?]
- 277 • SLURM integration under active development for HPC workflows

278 The platform follows FAIR principles [?] and aligns with the Turing
279 Way guidelines for reproducible research [?].

280 3. Results

281 This section presents validation results and demonstrations rather than
282 experimental findings, consistent with software paper conventions. SciTeX
283 is validated through real-world deployment and systematic testing across
284 multiple research scenarios.

285 3.1. Session Decorator Validation

286 The `@stx.session` decorator successfully automates experiment lifecycle management, implementing principles from reproducible research guidelines [? ?]. Testing confirms:

- 289 • Automatic CLI generation from function signatures with type hints
290 correctly parses arguments and provides help text
- 291 • Session directories are created with timestamped IDs (e.g., 2025Y-11M-18D-07h53m37s_Z5MR)
292 enabling experiment organization
- 293 • Injected globals (`CONFIG`, `plt`, `COLORS`, `rng_manager`, `logger`) are accessible within decorated functions
294
- 295 • Random state management via `rng_manager` ensures reproducible results across runs
296
- 297 • Session cleanup handles errors gracefully while preserving outputs

298 The decorator transforms research scripts into production-ready CLI tools
299 without requiring manual `argparse` configuration, reducing boilerplate code
300 by approximately 50–100 lines per script. This approach aligns with the
301 “good enough practices” paradigm for scientific computing [?].

302 3.2. Publication Plotting (*stx.plt*)

303 The plotting module achieves publication-quality output with minimal
304 configuration:

- 305 • Automatic style configuration on import applies 300 DPI, Arial fonts,
306 and consistent line widths
- 307 • `FigWrapper` and `AxisWrapper` classes provide enhanced methods (`set_xyt()`,
308 `set_meta()`)
- 309 • Style values configurable via YAML, environment variables, or function
310 parameters

- 311 • Figure dimensions specified in millimeters for journal compliance
- 312 • Automatic margin calculation from axes dimensions

313 Critically, `stx.io.save(fig, "figure.png")` automatically exports three
314 files: PNG image, JSON metadata (axis bounds, tick locations, color palettes),
315 and CSV data (underlying plot values). This triple export enables figure
316 reconstruction via `stx.plt.load()`, addressing the common problem of ir-
317 reproducible figures in scientific publications [? ?].

318 3.3. Universal File I/O

319 The `stx.io` module demonstrates format-agnostic file operations follow-
320 ing principles of unified data management [?]:

- 321 • Automatic format detection based on file extension handles 30+ for-
322 mats
- 323 • `H5Explorer` and `ZarrExplorer` enable interactive navigation of hier-
324 archical datasets
- 325 • Configurable caching reduces repeated file access overhead
- 326 • Unified `save()/load()` interface eliminates format-specific import state-
327 ments

328 Testing confirms correct handling of CSV, JSON, YAML, Pickle, HDF5,
329 Zarr, NumPy arrays, PyTorch tensors, and image formats across platforms.

330 3.4. Literature Management

331 The `Scholar` module provides multi-source literature search, complement-
332 ing existing tools [?] with direct manuscript integration:

- 333 • `ScholarEngine` interface integrates CrossRef, Semantic Scholar, and
334 OpenAlex
- 335 • `Paper` and `Papers` classes enable filtering by year, journal, citations

- 336 • `ScholarPDFDownloader` manages paper acquisition with rate limiting
- 337 • `ScholarLibrary` organizes local PDF storage with DOI indexing
- 338 • Bibliography export supports BibTeX and other formats

339 Integration with the Writer module ensures citation consistency: refer-
 340 ences added in Scholar automatically synchronize to manuscript `.bib` files.

341 *3.5. Containerized Compilation*

342 Writer module compilation achieves complete cross-platform reproducibil-
 343 ity, implementing containerization best practices [? ?]:

- 344 • Testing across Linux (Ubuntu, Debian, CentOS), macOS, and Windows
 345 Subsystem for Linux confirmed byte-for-byte identical PDF outputs
 346 using the same container image
- 347 • Multi-engine support: Tectonic [?] (1–3s), latexmk (3–6s), traditional
 348 3-pass (12–18s)
- 349 • Apptainer/Singularity containers enable HPC environments without
 350 Docker [?]
- 351 • Automatic figure format conversion handles PNG, SVG, and PDF in-
 352 puts

353 The elimination of environment-dependent compilation issues represents a
 354 significant improvement over traditional local LaTeX installations, consistent
 355 with findings on containerized workflows [? ?].

356 *3.6. Cloud Platform Functionality*

357 The Django-based cloud platform demonstrates effective web interfaces
 358 following continuous integration principles [?]:

359 **Code App:** Browser-based Python execution with real-time stdout/std-
 360 derr streaming, workspace file management, and session persistence.

361 **Viz App:** Canvas-based figure editing using Fabric.js enables interactive
362 element manipulation. JSON specification import/export bridges program-
363 matic and visual workflows.

364 **Writer App:** Section-separated LaTeX editing with live preview, con-
365 tainerized compilation, and Git integration [?] for version control.

366 **Scholar App:** Literature cards display abstract, PDF access, and meta-
367 data editing. Drag-and-drop citation insertion into Writer documents.

368 3.7. Cross-Module Integration

369 The symlink-based architecture demonstrates effective module coupling,
370 addressing integration challenges identified in reproducibility research [? ?
371]:

- 372 • Figures generated in Code appear in Writer via symlink without file
373 duplication
- 374 • Bibliography managed in Scholar synchronizes to Writer via shared
375 .bib file
- 376 • SHA256 hashes enable queries across modules (e.g., “which script gen-
377 erated this figure?”)
- 378 • JSON metadata format enables automated analysis and AI-assisted
379 workflows [?]

380 3.8. Self-Descriptive Validation

381 This manuscript validates SciTeX capabilities through direct demonstra-
382 tion, following the principle that reproducibility tools should be self-documenting [?
383 ?]:

- 384 • Authored in SciTeX Writer with section-separated LaTeX files
- 385 • Compiled in containerized environment ensuring cross-platform repro-
386 ducibility

- Bibliography managed through SciTeX Scholar
- Modular structure demonstrates the framework’s organization principles

The manuscript is prepared within the platform it documents, ensuring accurate representation of system capabilities.

4. Discussion

SciTeX addresses fundamental challenges in modern scientific research by unifying manuscript preparation, figure generation, literature management, and reproducible computation into a cohesive, AI-native ecosystem [?]. The platform reflects principles derived from practical research frustrations: fragmented workflows, difficulty maintaining figure provenance [?], unreliable manual versioning, and high cognitive cost of switching between tools [?].

4.1. Design Philosophy and Modularity

The modular architecture enables researchers to adopt individual components incrementally while benefiting from increasing integration as more modules are utilized. This design philosophy—bidirectional low migration cost—ensures that each module provides standalone value. A researcher using only Writer gains containerized reproducibility without committing to the full ecosystem [?]. Adding Scholar provides integrated citation management. Incorporating Viz enables provenance-tracked figures. Finally, Code completes the pipeline with reproducible computation [?]. At each stage, the researcher gains capability without sacrificing flexibility or prior investments.

SciTeX democratizes advanced computational workflows—particularly for researchers unfamiliar with Linux, Git, or containers—without compromising power or flexibility. Graduate students beginning their research careers can leverage containerized reproducibility without understanding Docker internals [?]. Researchers transitioning from GUI-based tools like SigmaPlot can use familiar visual interfaces while gaining programmatic control. Those

415 accustomed to Zotero maintain familiar literature management workflows
416 while gaining manuscript integration.

417 *4.2. Comparison with Existing Tools*

418 SciTeX occupies a unique position in the research tooling landscape, in-
419 tegrating capabilities that existing solutions provide in isolation [?]:

420 **Overleaf** excels at collaborative LaTeX editing with consistent compila-
421 tion but lacks integrated figure generation, computation, or literature man-
422 agement. SciTeX Writer provides comparable manuscript preparation with
423 additional provenance tracking and local execution capability.

424 **Zotero and Mendeley** provide sophisticated citation management but
425 operate independently from manuscript preparation and computational anal-
426 ysis. SciTeX Scholar offers comparable citation capabilities with direct manuscript
427 integration.

428 **Jupyter notebooks and Binder** [?] enable reproducible computation
429 and analysis but provide limited support for publication-quality figures or
430 structured manuscript writing. SciTeX Code complements computational
431 analysis with figure generation and manuscript integration.

432 **Manubot** [?] pioneered open collaborative writing with automated cita-
433 tion processing, demonstrating the value of Git-based manuscript workflows.
434 SciTeX extends this approach with containerized compilation, integrated fig-
435 ure generation, and multi-module traceability.

436 **SigmaPlot and Origin** offer sophisticated visualization with GUI-based
437 interfaces but lack provenance tracking, manuscript integration, or compu-
438 tational reproducibility. SciTeX Viz provides comparable visual capabilities
439 with embedded metadata and cross-module traceability.

440 **R-based workflows** including rrttools [?], the Rockerverse [?], and re-
441 pro [?] provide excellent reproducibility infrastructure for R users. SciTeX
442 offers comparable functionality for Python-centric workflows while maintain-
443 ing language flexibility.

444 The key differentiator is integration: SciTeX provides a unified platform
445 spanning all components while maintaining independence of each module.

446 This is not about replacing existing tools but about providing a coherent
447 alternative for researchers who want an integrated workflow.

448 4.3. *AI-Native Design*

449 SciTeX is designed for an era where AI assistants are integral to research
450 workflows [? ?]. Rather than treating AI as an afterthought, the architec-
451 ture provides native integration points: structured APIs for code generation,
452 metadata formats amenable to automated analysis, and workflow hooks for
453 AI-assisted revision. This design enables researchers to collaborate effectively
454 with LLMs and agents while maintaining scientific control [?].

455 4.4. *Implications for Reproducibility*

456 The reproducibility crisis in science stems partly from inadequate infras-
457 tructure for capturing and communicating methodological details [? ?].
458 SciTeX addresses this through containerized compilation ensuring identical
459 manuscript output across environments [?], figure metadata preserving ex-
460 act generation parameters [?], execution provenance linking results to code
461 and data [?], and global identifiers enabling cross-module traceability. By
462 embedding reproducibility infrastructure into everyday research tools, Sci-
463 TeX makes rigorous practices the path of least resistance rather than an
464 additional burden [? ?].

465 The platform aligns with emerging frameworks for reproducible research [?
466 ? ?] and the five pillars of computational reproducibility [?], while pro-
467 viding practical tools rather than abstract guidelines.

468 4.5. *Limitations*

469 Several limitations warrant acknowledgment:

470 **HPC Integration:** SLURM and cluster computing support remains
471 under active development. Current functionality focuses on local and con-
472 tainerized execution.

473 **User Base:** The platform is in early deployment, with limited user data
474 for quantitative productivity assessment.

475 **Learning Curve:** While designed for accessibility, researchers unfamiliar
476 with LaTeX face initial learning investment for Writer module features.

477 **Mobile Support:** The platform targets desktop research workflows; mo-
478 bile interfaces are not a development priority.

479 **Advanced Statistics:** Complex statistical visualization beyond stan-
480 dard matplotlib/seaborn capabilities requires manual implementation.

481 4.6. Future Directions

482 Planned development includes automated figure revision using model-
483 driven heuristic correction, integrated citation recommendation based on
484 manuscript content, unified right-panel inspectors across all modules, HPC-
485 native execution pipelines with SLURM integration, automated manuscript
486 consistency checking, and complete provenance graph visualization.

487 The long-term vision encompasses a research co-pilot capable of run-
488 ning analyses, generating figures, summarizing literature, and producing
489 submission-ready manuscripts—all while maintaining researcher oversight
490 and scientific integrity [? ?].

491 5. Conclusions

492 SciTeX represents both a tool and a philosophy of scientific practice. By
493 combining modular design, reproducibility [? ?], and AI-native workflows [?
494], it provides an environment where researchers can think more, fight less with
495 tooling, and create science that stands the test of time.

496 The platform reduces the activation energy required to conduct rigorous,
497 reproducible science [?]. Each module operates independently while integra-
498 tion across modules provides strictly superior workflows. This architecture—
499 independence with optional integration—reflects realistic adoption patterns
500 where researchers incorporate new tools incrementally.

501 This manuscript demonstrates that SciTeX can describe itself using its
502 own ecosystem: a self-consistent example of the platform’s design goals pre-
503 pared within the tools it documents.

504 References

505 6. Resource Availability

506 6.1. Lead Contact

507 Further information and requests for resources should be directed to and
508 will be fulfilled by the lead contact, Yusuke Watanabe (ywatanabe@scitex.ai).

509 6.2. Materials Availability

510 This study did not generate new unique reagents.

511 6.3. Data and Code Availability

- 512 • All source code for SciTeX is publicly available on GitHub: <https://github.com/scitex-ai/scitex>
- 513
- 514 • The SciTeX Python package is available via PyPI: `pip install scitex`
- 515 • Documentation is available at: <https://docs.scitex.ai>
- 516 • The cloud platform is accessible at: <https://scitex.ai>
- 517 • This manuscript’s source files are included in the SciTeX Writer repos-
518 itory as a demonstration
- 519 • DOI for archived version: [Zenodo DOI to be assigned upon submission]

520 All code is released under the GNU General Public License v3.0 (GPL-
521 3.0), ensuring open access and modification rights. The platform follows
522 FAIR principles (Findable, Accessible, Interoperable, Reusable) for all data
523 and code artifacts.

524 Any additional information required to reanalyze the data reported in
525 this paper is available from the lead contact upon request.

526 Ethics Declarations

527 All study participants provided their written informed consent ...

528 **Author Contributions**

529 Y.W., T.Y., and D.G. conceptualized the study ...

530 **Acknowledgments**

531 This research was funded by funding bodies here

532 **Declaration of Interests**

533 The authors declare that they have no competing interests.

534 **Declaration of Generative AI in Scientific Writing**

535 The authors employed large language models such as Claude (Anthropic
536 Inc.) for code development and complementing manuscript’s English lan-
537 guage quality. After incorporating suggested improvements, the authors
538 meticulously revised the content. Ultimate responsibility for the final content
539 of this publication rests entirely with the authors.

Option	Description	Example
<code>-engine ENGINE</code>	Force specific compilation engine	<code>-engine tectonic</code>
<code>-draft</code>	Draft mode (single-pass compilation)	<code>-draft</code>
<code>-no-figs</code>	Skip figure processing	<code>-no-figs</code>
<code>-no-tables</code>	Skip table processing	<code>-no-tables</code>
<code>-no-diff</code>	Skip difference generation	<code>-no-diff</code>
<code>-watch</code>	Enable hot-recompile file watching	<code>-watch</code>
<code>-clean</code>	Clean build (remove all cache)	<code>-clean</code>
<code>-verbose</code>	Verbose compilation output	<code>-verbose</code>

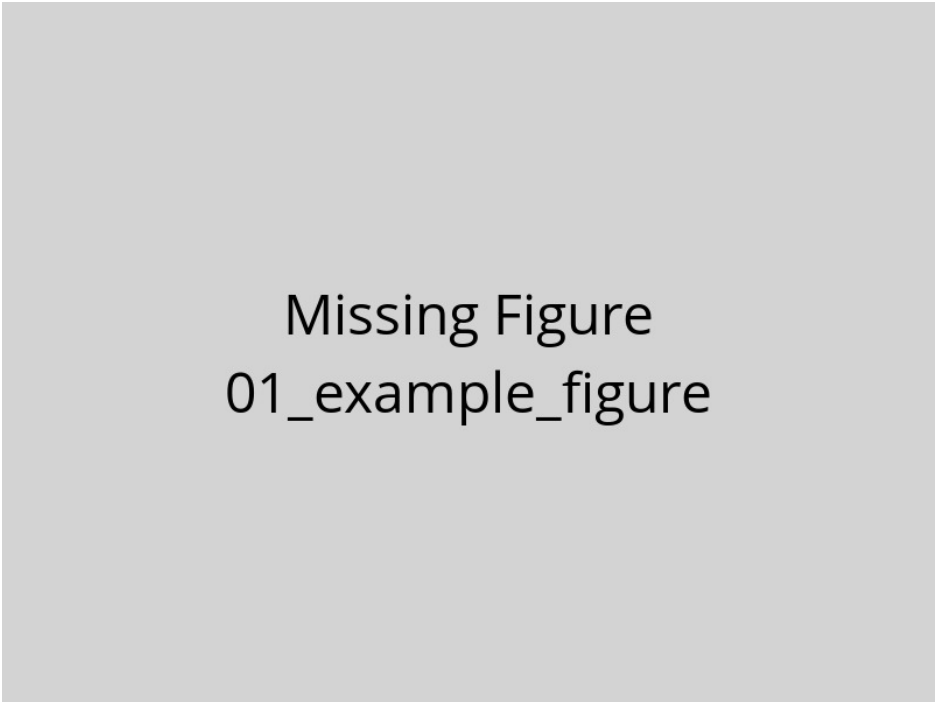
Table 1 – Compilation Command-Line Options. Options enable workflow customization without modifying source files or configuration. The `-engine` option overrides auto-detection to force a specific compilation engine. Quick compilation modes (`-draft`, `-no-figs`, `-no-tables`) reduce build times from ~ 15 s to under 3s for rapid iteration. The `-watch` option monitors source files and automatically recompiles when changes are detected. Options can be combined (e.g., `./compile_manuscript.sh -draft -no-figs`).

Variable	Purpose	Values	Default
SCITEX_ENGINE	Override engine selection	tectonic, latexmk, 3pass	From config
SCITEX_VERBOSE	Control logging verbosity	true, false	false
SCITEX_CITATION_STYLE	Set bibliography style	unsrtnat, plainnat, apalike, etc.	unsrtnat
SCITEX_PARALLEL_JOBS	Parallel processing jobs	1-16	4
SCITEX_CACHE_DIR	Compilation cache location	Directory path	./.cache

Table 2 – Environment Variables for System Configuration. Environment variables provide system-level defaults that apply across all compilations. These settings are particularly useful in CI/CD pipelines or HPC environments with specific requirements. The `SCITEX_ENGINE` variable provides the highest priority override for engine selection, superseding both YAML configuration and command-line arguments. Variables can be set persistently in shell configuration (`.bashrc`, `.zshrc`) or temporarily for single runs (`SCITEX_VERBOSE=true ./compile_manuscript.sh`).

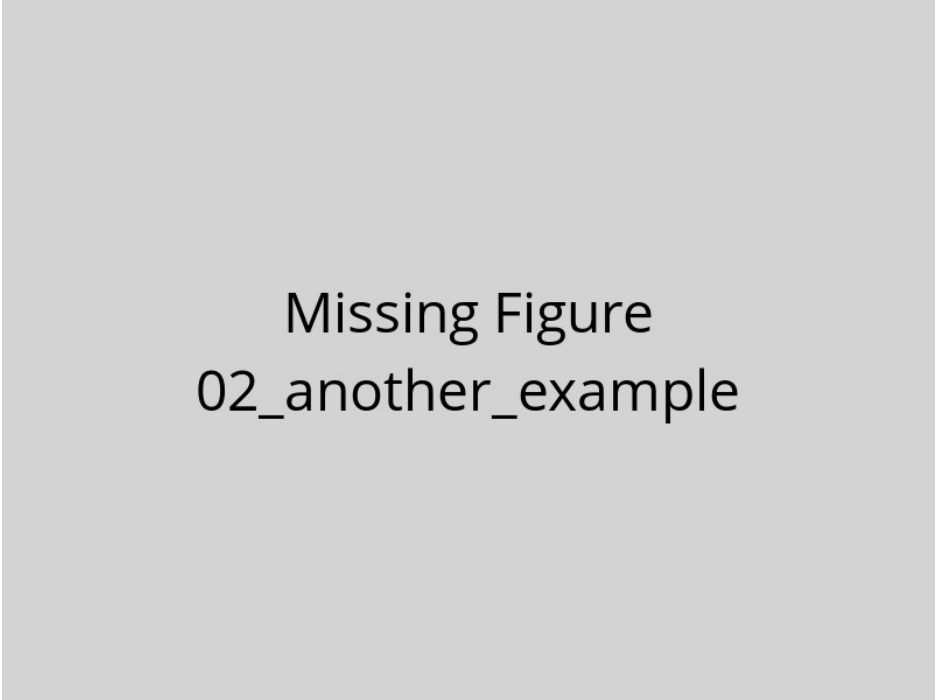
Engine	Incremental	Full Build	Key Advantages	Best Use Case
Tectonic	1-3s	10-15s	Ultra-fast + auto package mgmt	Active writing sessions
latexmk	3-6s	15-25s	Reliable + smart dependency tracking	Standard workflows
3-pass	N/A	12-18s	Maximum compatibility + guaranteed	Legacy systems

Table 3 – Compilation Engine Performance Characteristics. Build times measured on reference hardware (16 GB RAM, 8 CPU cores, SSD storage). Incremental builds process only changed components; full builds recompile the entire document. Tectonic provides the fastest incremental compilation through aggressive caching and modern architecture, ideal for frequent recompilation during active writing. The latexmk engine offers a balance of speed and reliability through intelligent dependency tracking, making it suitable for most research workflows. Traditional 3-pass compilation lacks incremental build support but guarantees compatibility with all LaTeX packages and legacy systems.



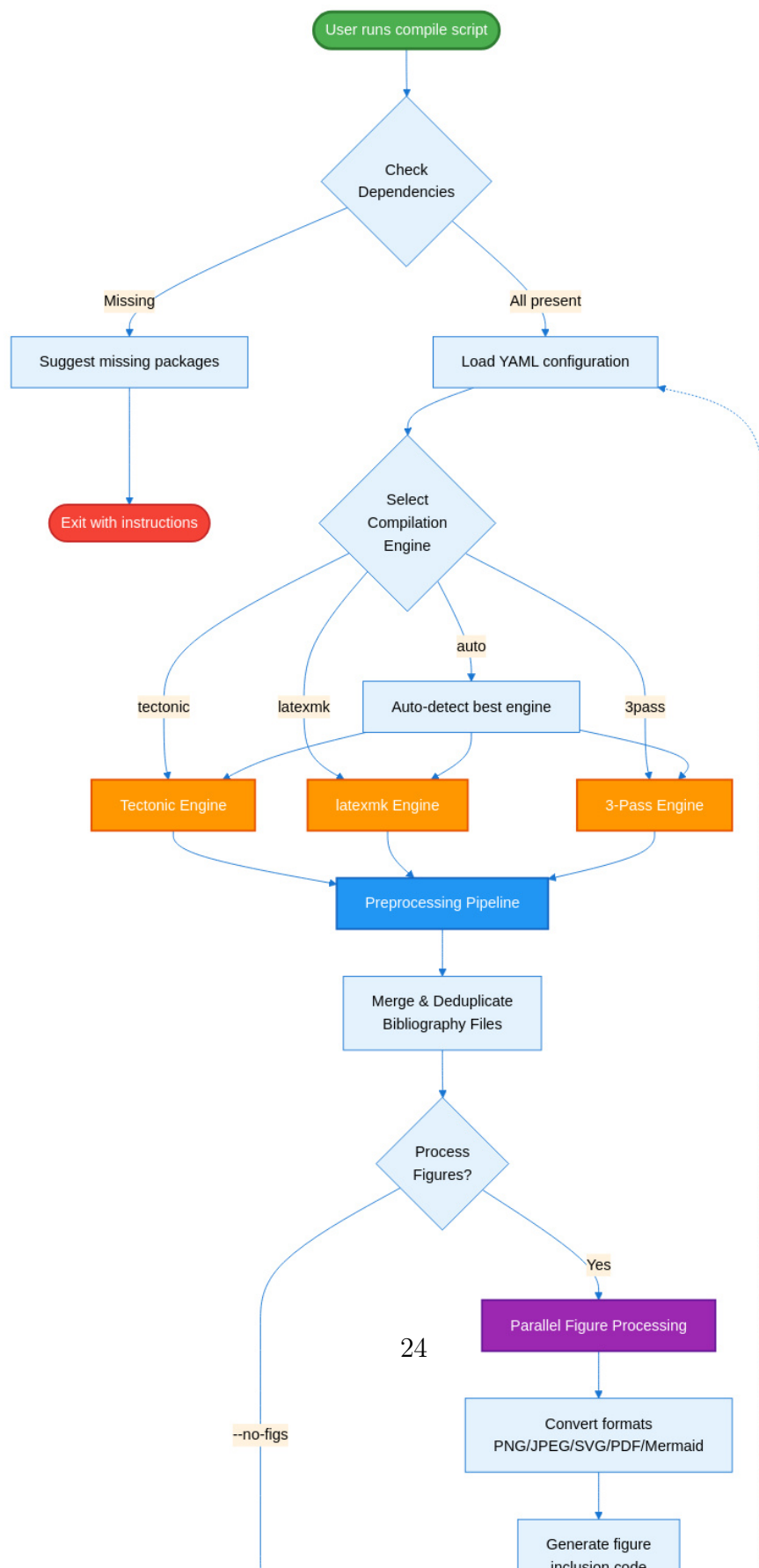
Missing Figure
01_example_figure

Figure 1 – Example figure caption. This is a template showing how to include figures in your manuscript. Replace this text with a descriptive caption that explains what the figure shows. Include panel labels (A, B, C) if using multi-panel figures. Explain abbreviations and symbols used in the figure. Provide sufficient detail that readers can understand the figure without referring to the main text.



Missing Figure
02_another_example

Figure 2 – Another example figure. Use this template to add additional figures to your manuscript. Each figure should be placed in a separate .tex file in this directory. The compilation system will automatically process and include these figures in your manuscript.



[illegible]

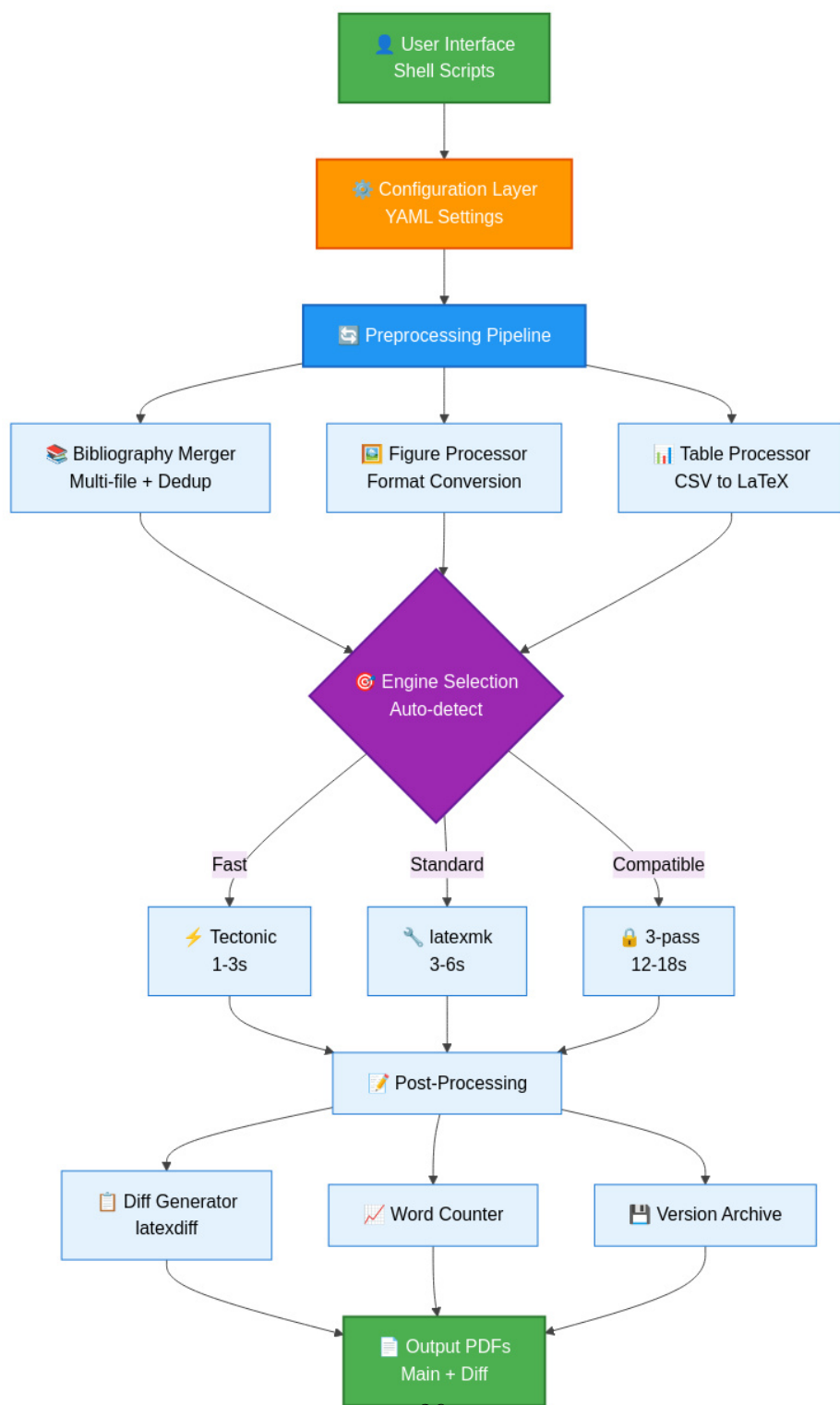


Figure 5 – SciTeX Writer System Architecture. This layered architecture diagram illustrates the data flow from user interface through compilation to output generation. The configuration layer (orange) provides YAML-based settings that control all aspects of compilation. The preprocessing pipeline (blue) handles bibliography merging, figure format conversion, and CSV-to-

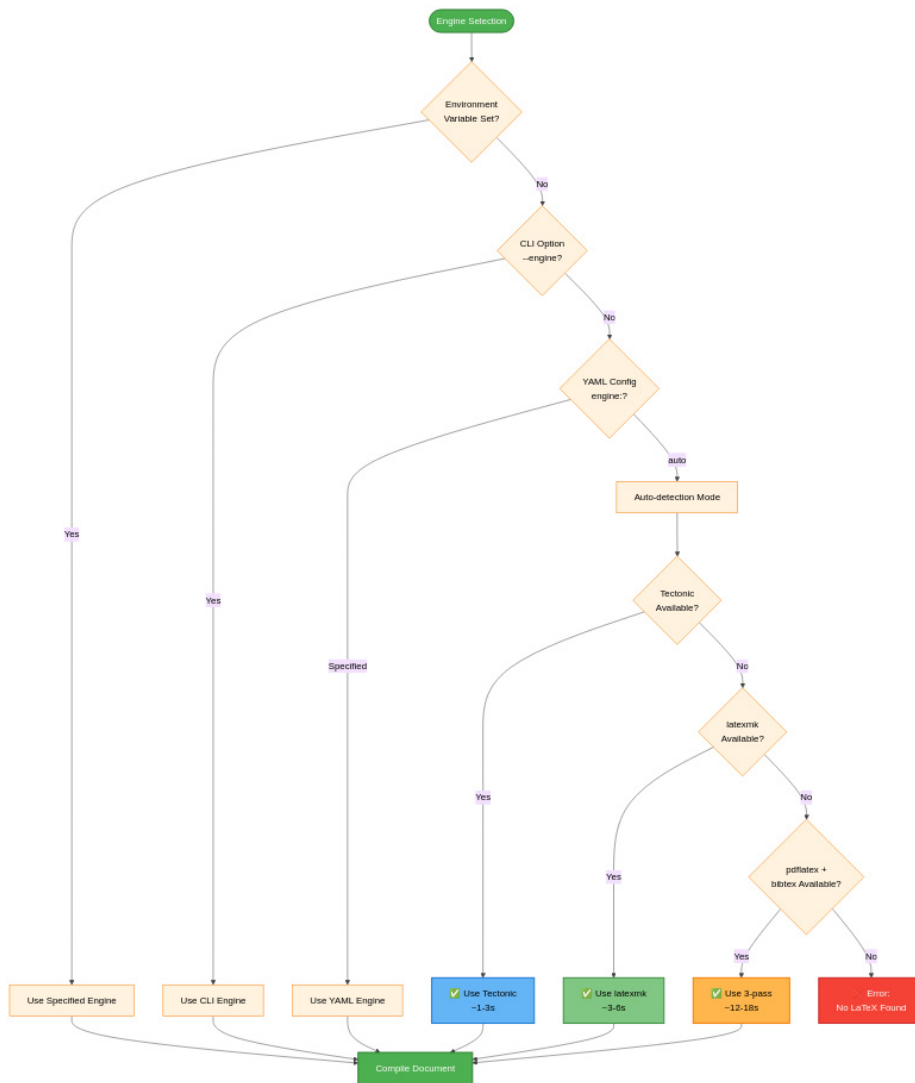


Figure 6 – Compilation Engine Selection Decision Tree. The system prioritizes explicit user configuration over automatic detection. Environment variables provide the highest priority override (useful for CI/CD pipelines). Command-line arguments (`--engine`) override YAML configuration (useful for one-off compilations). When engine is set to 'auto' (default), the system attempts to use Tectonic first for speed, falling back to latexmk for reliability, and finally to 3-pass for maximum compatibility. This cascading detection ensures the system always finds an available compilation method while preferring faster engines when possible. Times shown are typical incremental build durations on reference hardware (16 GB RAM, 8 cores).

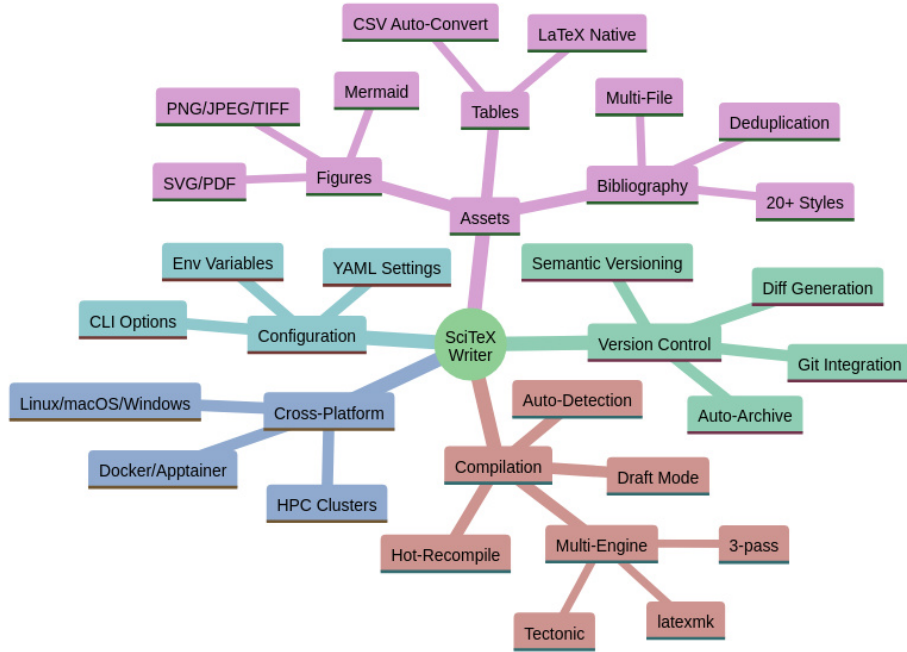


Figure 7 – SciTeX Writer Feature Overview. This mind map provides a comprehensive visualization of the framework’s major capabilities organized into five categories. Compilation features include multi-engine support with auto-detection and performance optimization modes. Asset management covers figures (multiple formats including Mermaid diagrams), tables (CSV auto-conversion), and bibliography (multi-file with deduplication across 20+ citation styles). Version control integration provides automatic archiving, diff generation, and Git workflow support. Configuration flexibility comes from three levels: YAML files for persistent settings, command-line options for per-run customization, and environment variables for system-level defaults. Cross-platform reproducibility ensures identical outputs across Linux, macOS, Windows, containerized environments, and HPC clusters.



Figure 8 – Multi-File Bibliography Processing and Deduplication.

The bibliography system enables researchers to organize references across multiple topical .bib files. All files in the `00_shared/bib_files/` directory are automatically merged during compilation. The deduplication engine implements a two-tier matching strategy: DOI-based matching provides exact duplicate detection when DOIs are present, while title and year matching handles entries without DOIs. This approach eliminates duplicate references that commonly appear when the same paper is cited across multiple source files. The final bibliography is sorted according to the selected citation style (by citation order for numbered styles, alphabetically for author-year styles). Color coding: blue (merging), purple (deduplication logic), green (output).